SOFTWARE ENGINEERING PROJECT

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year 2026

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

A Project Submitted in Partial Fulfillment of the Requirement for the Degree of Bachelor of Engineering (Software and Knowledge Engineering)
Department of Computer Engineering
Faculty of Engineering
KASERTSART UNIVERSITY
Academic Year 2026

Approved By:

Advisor . . . . . **Date** . . . . . .
(Project Advisor's Name)

Co-Advisor . . . . . **Date** . . . . . .
(Project Co-Advisor's Name)

Head of Department . . . . . **Date** . . . . . .
(Head of Department's Name)

Abstract

Put your abstract paragraph here.

Acknowledgement

Put your acknowledgement paragraph here.

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

Table of Contents

Content	Page
Abstract	**i**
Acknowledgement	**ii**
List of Tables	**v**
List of Figures	**vi**
Chapter 1 Introduction	**1**
1.1 Background	1
1.2 Problem Statement	1
1.3 Solution Overview	2
1.3.1 Features	3
1.4 Target User	3
1.5 Terminology	3
Chapter 2 Literature Review and Related Work	**5**
2.1 Competitor Analysis	5
2.2 Literature Review	5
Chapter 3 Requirement Analysis	**7**
3.1 Stakeholder Analysis	7
3.2 User Stories and Use Cases	7
3.2.1 Student	7
3.2.2 Software Developer	8
3.2.3 Scrum Master	8
3.3 Use Case Diagrams	8

3.4 Use Case Model 9
3.5 User Interface Design 10

Chapter 4 Software Architecture Design 11
4.1 System Architecture Overview 11
4.2 Software Design 11
4.3 AI and Data Design 11
4.3.1 Data Sources and Collection 11
4.3.2 Model Design 12
4.3.3 Evaluation 12

Chapter 5 Software Development 13
5.1 Software Development Methodology 13
5.2 Technology Stack 13
5.3 Coding Standards 14
5.4 Schedule 14
5.5 Progress Tracking Report 14

Chapter 6 Deliverables 15
6.1 Software Solution 15
6.2 Test Report 15

Chapter 7 Conclusion and Discussion 16
7.1 Summary of Achievements 16
7.2 Known Limitations 16
7.3 Future Work 16

Appendix A: Example 20

Appendix B: About LaTeX 22

List of Tables

Page

List of Figures

Page

2.1	Competitive Landscape by Asana	5
3.1	User Interface Design	10
5.1	Example technology stack	13

Chapter 1

Introduction

1.1 Background

Meetings are an indispensable tool for collaborative work in both academic and professional settings. In theory, a focused meeting covering a single topic should require no more than 15–30 minutes. In practice, however, teams routinely turn short discussions into multi-hour sessions. The root cause is *topic drift*: the gradual, often unnoticed, departure of conversation from the original agenda.

The development team behind MeetRX experienced this problem first-hand during group project work. What should have been a concise weekly sync repeatedly stretched into a three-hour session because members lost track of the intended focus and the conversation wandered across unrelated topics. This observation motivated the creation of an intelligent meeting assistant that can detect drift as it occurs and help teams stay aligned.

1.2 Problem Statement

The following problems were identified through the team's own experience and stakeholder interviews:

1. **Burden on the minute-taker.** During meetings, the person responsible for taking notes cannot participate meaningfully in the discussion. Their attention is divided between listening, comprehending, and writing, which reduces both the quality of the notes and their contribution to the meeting.

2. **Standup meetings overrunning their allotted time.** Daily standups and progress-update meetings are intended to be short, time-boxed events. In practice, they frequently exceed their scheduled duration because participants introduce tangential topics, triggering lengthy discussions that should be handled separately.

3. **Repeated debates caused by poor institutional memory.** In long-term projects with frequent meetings, teams often forget what was discussed or decided in earlier sessions. As a result, the same topics are re-opened and the same decisions are re-debated, consuming time that could be spent on new work.

1.3 Solution Overview

MeetRX addresses the problems above through three integrated AI subsystems:

1. **Summarisation AI**: Automatically transcribes meeting audio using OpenAI Whisper and generates a structured summary so that no human minute-taker is required. All participants can engage fully in the discussion.

2. **Topic Drift Detection AI (Main Feature)**: Continuously analyses the live conversation using natural language processing techniques to assign a drift score to recent utterances. When the score exceeds a configurable threshold, a large language model re-evaluation is triggered and a real-time alert is surfaced to meeting participants, prompting them to return to the agenda.

3. **Agentic AI**: Provides a conversational interface that can answer questions about past meetings, look up decisions recorded in previous summaries, query the project codebase, and integrate with external tools to retrieve relevant context on demand.

1.3.1 Features

1. **Real-time Topic Drift Detection**: Monitors conversation flow and raises an alert when discussion deviates from the stated agenda topic.
2. **Automatic Meeting Summarisation**: Produces a concise, structured summary at the end of every session without requiring manual note-taking.
3. **Q&A Chatbot**: Allows participants to query past meeting summaries and project context using natural language.
4. **Task Assignment**: Extracts and records action items from meeting transcripts and assigns them to the appropriate team members.

1.4 Target User

MeetRX is designed for three primary user groups:

- **Students**: Particularly those conducting group project meetings who need help keeping discussions on-topic and producing reliable records of decisions made.
- **Software Developers**: Engineering teams running daily standups, sprint planning, and retrospectives who want shorter, more focused meetings and automatic summaries.
- **Scrum Masters**: Facilitators responsible for sprint ceremonies who need tooling to enforce time-boxes and surface relevant historical context during discussions.

1.5 Terminology

Topic Drift A situation in which the ongoing conversation is no longer contributing to the intended meeting topic or agenda item.

Drift Score A numerical value computed by the NLP pipeline that quantifies how far the current utterance has deviated from the baseline topic embedding.

Agenda A predefined list of topics that a meeting is intended to cover.

Utterance A single spoken turn or message produced by one participant during a meeting.

Summarisation The process of condensing a meeting transcript into a shorter document that captures the key points, decisions, and action items.

Agentic AI An AI system capable of autonomously invoking external tools or data sources to answer queries without explicit step-by-step instructions from the user.

LLM Large Language Model; a neural network trained on large text corpora capable of generating and understanding natural language.

Whisper An open-source automatic speech recognition model developed by OpenAI.

Chapter 2

Literature Review and Related Work

2.1 Competitor Analysis

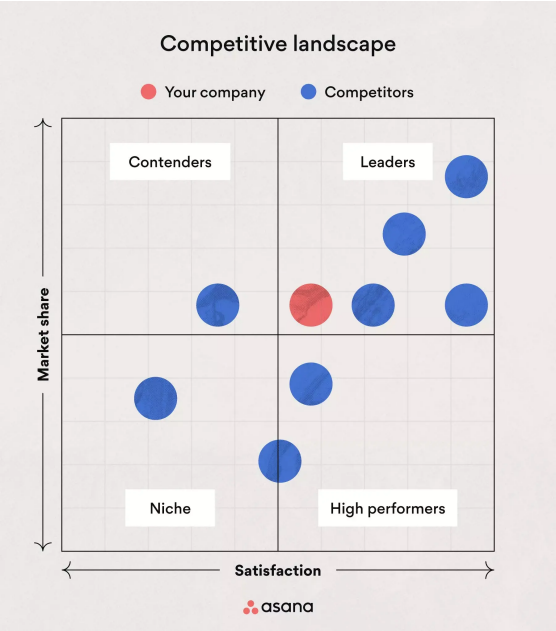

Figure 2.1: Competitive Landscape by Asana

<TIP: Refer to an article "How to create a competitive analysis (with examples)" by Asana. You can use the Competitor Landscape or Competitor Analysis Framework for your project. />

2.2 Literature Review

<TIP: Review academic papers or technical reports that are relevant to your project. For each source, briefly summarize its key findings and

then explain how it informs your own design or implementation decisions. The goal is not just to describe what others have done, but to build a justification for your own technical choices. For example, if you chose a particular algorithm or architecture, cite the work that supports that decision. />

Chapter 3

Requirement Analysis

3.1 Stakeholder Analysis

Students University students working in group projects need a tool that detects when their meeting drifts off the project agenda and automatically generates meeting minutes so that all members can participate in the discussion rather than taking turns as note-taker.

Software Developers (Office Workers) Software engineers attending daily standups and sprint ceremonies benefit from automatic meeting summaries and from an agent that can surface relevant decisions from past sessions, reducing repeated debate.

Scrum Masters / Project Managers Facilitators need visibility into whether a meeting is staying on-topic and access to a historical record of decisions so they can run tighter, more accountable ceremonies.

3.2 User Stories and Use Cases

3.2.1 Student

1. As a **student**, I want the system to alert me when my group's conversation goes off-topic, so that we can finish our meeting within the scheduled time.
2. As a **student**, I want an automatic summary of each meeting, so that I do not have to assign a dedicated note-taker.
3. As a **student**, I want to query what was decided in a previous meeting, so that I do not have to re-read full transcripts.

3.2.2 Software Developer

1. As a **software developer**, I want standup meetings to stay focused on project progress, so that they finish within 15 minutes.
2. As a **software developer**, I want action items extracted automatically from the meeting transcript and assigned to the correct team member, so that nothing falls through the cracks.
3. As a **software developer**, I want to ask the AI agent about a past architectural decision, so that I do not have to search through old chat logs or documents.

3.2.3 Scrum Master

1. As a **scrum master**, I want a real-time indicator that shows whether the current discussion is on-topic, so that I can redirect the team without interrupting the conversation flow.
2. As a **scrum master**, I want all sprint summaries stored and indexed, so that I can reference them during retrospectives.

3.3 Use Case Diagrams

<TIP: Include a UML Use Case Diagram that shows all actors and their relationships to the use cases you described. Also provide the written description for each use case. />

3.4 Use Case Model

UC-01: Start Meeting

Actor	Meeting Participant
Precondition	The participant is authenticated and has a meeting agenda prepared.
Main Flow	1. Participant opens the MeetRX dashboard. 2. Participant clicks “Start Meeting” and enters agenda topics. 3. System begins audio capture and topic embedding initialisation. 4. Meeting session is created and all participants are notified.
Postcondition	An active meeting session exists with an initialised topic baseline.

UC-02: Receive Topic Drift Alert

Actor	Meeting Participant (all)
Precondition	A meeting session is active.
Main Flow	1. NLP module computes drift score for each utterance. 2. If score exceeds threshold, LLM re-evaluation is triggered. 3. System displays a drift alert banner visible to all participants. 4. Participant acknowledges the alert and guides conversation back to agenda.
Alternative Flow	If the participant dismisses the alert, the system logs the dismissal and resumes monitoring.
Postcondition	Alert is logged; meeting continues.

UC-03: Query Past Meetings

Actor	Meeting Participant
Precondition	At least one completed meeting summary exists in the system.
Main Flow	1. Participant opens the Q&A interface. 2. Participant types a natural language question. 3. Agentic AI retrieves relevant summaries and generates an answer. 4. System displays the answer with source meeting references.
Postcondition	Participant receives an answer grounded in historical meeting data.

3.5 User Interface Design

<TIP: Put the initial design of your application here. You can showcase a detailed design of a specific page or a sitemap of your application. See an example below./>

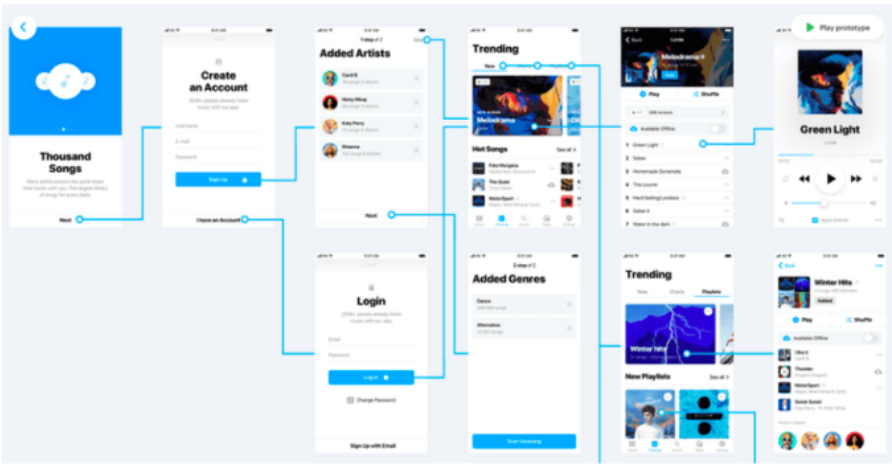

Figure 3.1: User Interface Design

Chapter 4

Software Architecture Design

4.1 System Architecture Overview

<TIP: Provide a high-level description of your system's structure. Include either a Domain Model (to describe core business concepts and their relationships) or a Design Class Diagram (to describe how your code is organized into classes and packages), whichever best communicates your design. Accompany your diagram with a short paragraph explaining the key components and how they interact. />

4.2 Software Design

<TIP: Describe the runtime behavior of your system. Include a Sequence Diagram for your most important use cases. Focus on scenarios with non-trivial interactions between components. If your project involves a novel algorithm, describe it here and explain why you designed it this way. />

4.3 AI and Data Design

4.3.1 Data Sources and Collection

<TIP: Describe where your data comes from. Is it collected from users, scraped from the web, sourced from a public dataset, or provided by an external API? Explain how much data you have or plan to collect, and any challenges you anticipate in obtaining it. Explain how data is cleaned, transformed, or prepared before use. />

4.3.2 Model Design

<TIP: Describe the AI model or external AI service you plan to use. Explain why you chose it over alternatives. This can connect back to your competitor or literature review in Chapter 2. If you are training your own model, describe the architecture and training approach. If you are using a third-party model or API, describe how it is integrated into your system and any prompt design or fine-tuning involved. />

4.3.3 Evaluation

<TIP: Describe the metrics you plan to use to evaluate your AI model. />

Chapter 5

Software Development

5.1 Software Development Methodology

<TIP: Describe your software development methodology in this section. />

5.2 Technology Stack

<TIP: Describe your technology stack here. See the following example from ThaiProgrammer.org />

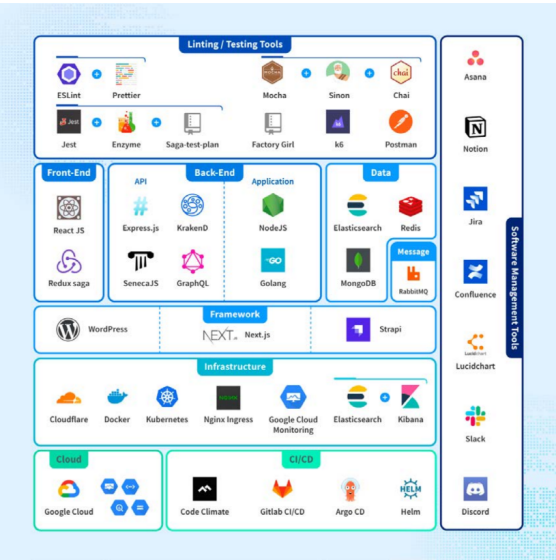

Figure 5.1: Example technology stack

5.3 Coding Standards

<TIP: Describe your coding standard for this project here. />

5.4 Schedule

<TIP: Show the schedule of your project. Use a Gantt chart. The items should agree with features you listed in Chapter 1. />

5.5 Progress Tracking Report

<TIP: Show that you have been working on this project overtime. It can be in the form of a burndown chart or a contribution graph from GitHub./>

Chapter 6

Deliverables

6.1 Software Solution

<TIP: Share a link to your Github repository. Showcase screenshots of the application and briefly describe each page here. />

6.2 Test Report

<TIP: Describe how you test your project. Place a test report here. If you use continuous integration and deployment (CI/CD) tools, describe your CI/CD method here. />

Chapter 7

Conclusion and Discussion

7.1 Summary of Achievements

<TIP: Summarize what your project accomplished. Refer back to your problem statement from Chapter 1 and explain the degree to which your solution addressed it. Highlight the most significant features or results delivered. />

7.2 Known Limitations

<TIP: Describe the current limitations of your solution honestly. These may include technical constraints (e.g., scalability, performance), scope limitations (features not yet implemented), data limitations (e.g., small training sets, biased data), or usability issues discovered during testing. />

7.3 Future Work

<TIP: Propose concrete improvements or extensions that could be made to this project given more time or resources. These should follow logically from your limitations. For example, if your model was trained on limited data, future work might include collecting a larger dataset. Avoid vague suggestions like "improve the UI"; instead, be specific about what would be changed and why. />

Reference

Bibliography

[1] Overleaf, “Learn latex in 30 minutes,” https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes.

Appendix A

Appendix A: Example

<TIP: Put additional or supplementary information/data/figures in appendices. />

Appendix B

Appendix B: About LaTeX

LaTeX (stylized as LaTeX) is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references.

LaTeX is widely used in academia for the communication and publication of scientific documents and technical note-taking in many fields, owing partially to its support for complex mathematical notation. It also has a prominent role in the preparation and publication of books and articles that contain complex multilingual materials, such as Arabic and Greek.

Overleaf has also provided a 30-minute guide on how you can get started on using LaTeX. [1]